Intro to Object Inheritance

(https://github.com/learn-co-curriculum/python-p3-intro-to-object-inheritance) (https://github.com/learn-co-curriculum/python-p3-intro-to-object-inheritance/issues/new)

Learning Goals

- Reduce repeated code and enhance objects using inheritance.
- Accomplish complex programming tasks using knowledge from previous modules.

Key Vocab

- **Inheritance**: a tool that allows us to recycle code by creating a class that "inherits" the attributes and methods of a parent class.
- **Composition**: a tool that enables you to recycle code by adding objects to other objects. Rather than building on a base class as in inheritance, composition leverages the attributes and methods of an instance of another class.
- **Subclass**: a class that inherits from another class. Colloquially called a "child" class.
- **Superclass**: a class that is inherited by another class. Colloquially called a "parent" class.
- **Child**: another name for a subclass.
- **Parent**: another name for a superclass.
- `super()` : a built-in Python function that allows us to manipulate the attributes and methods of a superclass from the body of its subclass.
- **Decorator**: syntax that allows us to add functionality to an object without modifying its structure.

Introduction

The concept of inheritance in Python works similarly to the real world — A prince can inherit a kingdom and everything within it; a baby can inherit genetic traits from a parent. In Python, a class can inherit the behaviors of another class, referred to as the ***superclass***.

In these lessons, we'll cover:

- What **inheritance** is in object-oriented Python.
- Implementing classes with inherited methods from another class that also has its own unique methods.
- Refactoring "code smells" into multiple, non-repetitive methods.
- Wrap functions and modify their behavior with **decorators**.
- Using the `super` keyword to inherit from and augment methods in the parent class.

In this section, we'll explain how we can leverage the power of Python to define basic classes with large reusability and smaller **subclasses** for more fine-grained, detailed behaviors.

Resources

- **Python 3.8 Documentation (https://docs.python.org/3.8/)**
- **Inheritance - Python (https://docs.python.org/3/tutorial/classes.html#inheritance)**
- **PEP 318 - Decorators for Functions and Methods (https://peps.python.org/pep-0318/)**
- **Inheritance and Composition: A Python OOP Guide - Real Python (https://realpython.com/inheritance-composition-python/)**
- **Decorators in Python - GeeksforGeeks (https://www.geeksforgeeks.org/decorators-in-python/)**
- **Supercharge Your Classes With Python super() - Real Python (https://realpython.com/python-super/)**